

M1.1: Tokio Work-Stealing Queue

Theory: Multi-threaded runtime minimizing lock contention. Each worker maintains a static 256-capacity local queue and a shared, mutex-guarded global queue.

Details: Workers prioritize local queues. If empty, a worker attempts to CAS-steal half the tasks of a busy worker's queue, failing which it checks the global queue. To prevent global queue starvation, workers check the global queue once every 61 loop ticks.

Trade-off: The static local queue size avoids allocator overheads but pushes burst traffic overflows into the locked global queue, introducing temporary lock contention.

M1.3: Coop Task Budget

Theory: Enforces scheduling fairness and prevents task starvation in cooperative multi-task systems.

Details: Every task is assigned a local budget of 128 work units per poll cycle. Async operations (network reads, channel pushes) decrement the budget. When it hits 0, the future returns 'Pending' and is re-queued.

Trade-off: Relies on coop-compatible APIs. Heavy CPU computations (e.g. matrix multiplication) without await yield gates can block the thread, requiring manual 'yield_now' or 'spawn_blocking' wrappers.

M1.5: Single-Threaded vs Multi-Threaded Schedulers

Theory: Supports 'current_thread' (single-threaded event loop) and 'multi_thread' (work-stealing pool) execute modes.

Details: 'current_thread' runs all tasks on the caller thread, avoiding thread migration, CAS synchronization, and 'Send' constraints; 'multi_thread' scales tasks across all available CPU cores.

Trade-off: 'current_thread' is ideal for low-latency, single-core services. 'multi_thread' offers massive overall throughput but requires tasks to be 'Send + Sync'.

M2.2: Token Registration & Map Routing

Theory: Routes ready events from millions of file descriptors to matching task wakers without global locking.

Details: Registering sockets generates a unique 64-bit Token, which maps to a slab array slot containing read/write task Waker pointers. Ready events wake up tasks via their registered pointers.

Trade-off: High-frequency registrations and deregistrations create memory allocation pressure on the slab array, which is mitigated via partitioned atomic operations.

M1.2: LIFO Slot Optimization

Theory: Accelerates task execution and optimizes CPU L1/L2 cache locality by prioritizing the most recently unparked task.

Details: Newly awoken tasks are written directly into a dedicated, single-element LIFO slot instead of the queue tail. The worker thread checks the LIFO slot first before looking at the run queue.

Trade-off: The LIFO slot can cause short-term task starvation in the circular queue. Tokio mitigates this by restricting the LIFO bypass to a single consecutive turn.

M1.4: Worker Thread Spawn & Sleep

Theory: Balance execution throughput and idle CPU power draw by dynamically parking and unparking threads.

Details: Workers communicate via atomic states (Searching, Idle). Idle workers park themselves via 'futex' when queues are empty. Searching threads are capped at half the maximum CPU cores to avoid bus thrashing.

Trade-off: Unparking a sleeping thread adds microsecond-level system call context-switch overhead, which can hurt latency during sudden traffic bursts.

M2.1: Mio Event Loop Integration

Theory: Standardizes platform-native non-blocking network socket APIs under a unified polling abstraction.

Details: Mio maps OS multiplexing APIs (Linux epoll, macOS kqueue, Windows IOCP) into 'mio::Poll'. The Tokio I/O driver executes Mio polling inside worker park loops.

Trade-off: Mio uses Edge-Triggered mode on Linux by default. Sockets must be read exhaustively until 'WouldBlock' is returned, otherwise the task will stall as no further ready events will fire.

M2.3: Park & Unpark Driver Integration

Theory: Eliminates dedicated thread overhead by executing I/O polls inside worker thread park loops.

Details: When a worker has no active tasks, it parks. If it is designated as the active driver, it calls 'epoll_wait' with a timeout determined by the next timer event, waking up when network events or timeouts occur.

Trade-off: If all worker threads are busy running tasks (none are parked), I/O events can experience processing delays. Tokio works around this with fast-path checks.

M3.1: Hashed Wheel Timer

Theory: Chronological timer scheduler running at $O(1)$ complexity to support millions of concurrent timeouts.

Details: Organized as a circular ring array of slots, each representing a fixed millisecond tick interval (default 1ms). Each slot holds a doubly linked list of registered timers.

Trade-off: Clock resolution is bounded by the tick size (1ms). High-precision nanosecond timings are not supported, and tick polling adds background CPU overhead.

M3.4: DelayQueue Scheduler

Theory: An asynchronous queue that holds elements until their individual delays expire.

Details: Backed by a binary heap and a hashed wheel timer. Insertion sorts items in the heap; when polled, the queue returns expired items, setting the sleep timeout of the next worker park to the next deadline.

Trade-off: Priority heap insertion takes $O(\log N)$ time. Mass insertions of elements with highly varied expirations can create CPU overhead during heap rebalancing.

M3.5: Time Driver Thread Integration

Theory: Coordinates time ticks alongside network events inside the main event loop.

Details: During worker park steps, the time driver determines the sleep interval based on the nearest timer expiration, using this value as the timeout parameter for 'epoll_wait'.

Trade-off: If a worker runs a CPU-bound task without yielding, the time driver is delayed, causing timer trigger latency.

M2.4: OS Multiplexing & IOCP Readiness Emulation

Theory: Bridges readiness-based event models (Linux/macOS) and completion-based models (Windows).

Details: Linux 'epoll' reports when socket buffers are ready for read/write. Windows 'IOCP' requires allocating buffers first, notifying users only after kernel operations complete.

Trade-off: Emulating readiness on IOCP requires Mio to maintain persistent in-memory buffers on Windows, introducing heap management and memory copying costs.

M3.2: Sleep & Timeout Futures

Theory: Standardized futures representing delayed execution or bounded execution times.

Details: Calling 'sleep(duration)' creates a 'TimerEntry' node and inserts it into the corresponding wheel slot, returning 'Pending'. The time driver wakes up the task when the wheel tick reaches the expiration mark.

Trade-off: Creating and cancelling timers frequently (e.g. for connection timeouts) creates heap allocation and linked list insertion overhead.

M4.1: Async Mutex

Theory: Asynchronous mutual exclusion lock that yields execution instead of blocking OS threads.

Details: If a task fails to acquire the lock, it inserts its Waker into a lock-internal queue and returns 'Pending'. When the owner releases the lock, it wakes up the next queued task.

Trade-off: Async locks incur significant heap management and waker coordination costs. Standard 'std::sync::Mutex' must be preferred unless the lock is held across 'await' boundaries.

M2.5: AsyncRead & AsyncWrite Polling

Theory: Non-blocking asynchronous I/O abstractions defining buffer reads and writes.

Details: 'poll_read' and 'poll_write' accept a Context reference. If a read/write would block, the task's Waker is registered to the socket's Token slot, and the call returns 'Poll::Pending'.

Trade-off: Waker consistency is critical. If a task returns 'Pending' but fails to update its registered waker, it will stall permanently when the socket becomes ready.

M3.3: Monotonic Clock

Theory: Guards against timer corruption caused by system clock adjustments or NTP clock drift.

Details: Tokio relies on the system monotonic clock (e.g., 'CLOCK_MONOTONIC'), which never drifts backward. Userspace reads are optimized via vDSO mapping to read CPU TSC registers.

Trade-off: Monotonic time lookups still add execution overhead, which Tokio optimizes by caching time values at the beginning of each scheduler loop.

M4.2: Async Semaphore

Theory: Asynchronous resource counting primitive limiting access to a pool of shared resources.

Details: Tasks decrement counts via CAS. If the count hits 0, tasks insert their nodes into a FIFO waiting queue. Dropping 'SemaphorePermit' automatically increments the count and wakes up the next queued node.

Trade-off: High contention on the wait queue tail pointers can lead to CPU branch prediction failures.

M4.3: Oneshot Cell

Theory: High-speed Single-Producer Single-Consumer (SPSC) channel designed for one-off values.

Details: Uses a single atomic State Cell. Senders write the value and CAS-transition the state to 'COMPLETE'. If empty, the receiver registers its Waker in the cell and yields 'Pending'.

Trade-off: Single-use only. Creating oneshot channels repeatedly in loops causes heap allocation churn.

M5.2: Watch Value Channel

Theory: Single-Producer Multi-Consumer (SPMC) channel that retains only the latest value.

Details: Holds a single value slot and an atomic version number. Senders increment the version; receivers yield 'Pending' if their local version matches the slot version.

Trade-off: Intermediate updates are discarded if the sender updates faster than receivers can poll.

M5.3: Join & Select Macro Multiplexing

Theory: Asynchronous control flow multiplexing. 'join' waits for all futures; 'select' returns as soon as the first future completes, dropping all other branches.

Details: 'select' randomizes branch polling order to ensure fairness and prevent branch starvation.

Trade-off: Unfinished branches are dropped, which can trigger cancellation safety issues if a dropped future was holding partial state.

M6.1: Tokio Console gRPC Telemetry

Theory: Real-time instrumentation tool for runtime profiling, deadlocks, and task latency.

Details: Emits events during spawn, poll, and drop cycles. 'console-subscriber' collects these events via ring buffers and exposes a gRPC dashboard endpoint.

Trade-off: Metric collection adds 3% to 5% CPU overhead, making it unsuitable for low-latency production pipelines.

M4.4: MPSC Ring Buffer

Theory: Multi-Producer Single-Consumer (MPSC) channel, serving as the communication base for async Actor designs.

Details: Senders concurrently push messages into a ring buffer using CAS operations. If bounded and full, senders are queued in a wait list to enforce backpressure.

Trade-off: Unbounded MPSC channels can cause out-of-memory crashes if consumption lags. Bounded channels block senders.

M5.4: Task Local Storage (TLS)

Theory: Binds variables to tasks rather than OS threads to support task migration across thread pools.

Details: Standard 'thread_local' breaks when tasks yield and resume on different worker threads. 'tokio::task_local' binds variables dynamically during the poll call.

Trade-off: Introduces lookup overhead on every read/write call during poll steps.

M6.2: Loom Concurrency Check

Theory: Permutation testing tool for lock-free concurrency verification.

Details: Loom executes concurrent code repeatedly, systematically permuting thread scheduling and atomic ordering states to detect data races or memory leaks.

Trade-off: Large state spaces cause exponential path explosion. Restrict tests to small, focused data structures.

M5.1: Broadcast Channel Lag Detection

Theory: Multi-Producer Multi-Consumer (MPMC) channel that broadcasts every message to all subscribers.

Details: Backed by a circular buffer. If a consumer reads too slowly, the write pointer will overwrite its unread slots, triggering a 'Lagged' error.

Trade-off: Sacrifices slow consumers to ensure fast consumers and senders are never blocked.

M5.5: Graceful Shutdown

Theory: Coordinates endpoint draining and task termination.

Details: Closes listening ports, broadcasts shutdown signals to active connections, and waits for active tasks to complete.

Trade-off: Infinite worker loops can hang shutdown steps. Configure a hard shutdown timeout limit to force-quit.

M6.4: Spawn Boxing Cost

Theory: Allocation trade-offs for dynamic scheduling.

Details: 'tokio::spawn' allocates the future on the heap (Box) and locks the scheduler queues to register the task.

Trade-off: Spawning adds O(1) allocation and locking cost. For small tasks, inline polling or nested select is faster than spawning separate threads.

Zone T1: Tokio APIs

tokio::spawn(), tokio::select(), tokio::join(),
tokio::task::yield_now(), tokio::sync::mpsc::channel(),
tokio::sync::Mutex::lock(), mio::Poll::poll()

Zone T2: System Execution Faults

thread_starvation_deadlock, stack_overflow_async, mem-
ory_leak_unbounded_mpsc, tcp_port_exhaustion_timewait,
timer_drift_jitter, cancellation_safety_lost_packet